

README: cdm_contract_model.py

この README は、cdm_contract_model.py を読む人が

1. CDM とは何か
2. この Python コードが CDM のどの思想を再現しているのか
3. 各クラスをどう読めばよいか
4. どう組み立てて使えばよいか
5. 何を意図的に省略しているのか

を順に理解できるように書いています。

1. これは何のコードか

cdm_contract_model.py は、**FINOS Common Domain Model (CDM)** の考え方を参考にして、**契約内容の表現**に重点を置いて作った Python モデルです。

一言でいうと、

- 取引は誰と誰のものか
- どんな商品なのか
- その商品の経済条件は何か
- どんな支払いルール (Payout) があるか
- その支払いルールに KO やデジタル条件のような条件付き feature があるか

を、Python の dataclass で表現するためのコードです。

このコードは **CDM の完全な実装ではありません**。

ただし、CDM のうち契約表現で重要な骨格をかなり意識して作ってあります。

2. まず CDM の思想をざっくり理解する

2.1 CDM は「契約を機械可読に表現する」ための共通モデル

CDM を知らない人向けにかなり噛み砕いていうと、CDM は

金融商品の契約内容を、システム同士で共通に理解できる形で表現するためのモデルです。

ここで大事なのは、CDM は単に「商品の名前」を持つモデルではないことです。

たとえば「5年の金利スワップ」と聞いても、実務では本当に必要なのは名前ではなく、

- 当事者
- 通貨
- ノーショナル
- 固定か変動か
- 利率やスプレッド
- 支払頻度
- リセット日
- 追加条件
- KO/KI などの条件付き条項

のような、**契約の中身そのもの**です。

CDM はこれを、なるべく分解された部品で表現しようとしています。

2.2 CDM は「将来の義務」を直接・間接に表現する

CDM の商品表現では、重要な考え方として **Payout** があります。

ざっくり言うと Payout は、

将来、どんな資産移転やキャッシュフローが発生するかを決めるルールです。

つまり CDM は、商品を

- 商品名のラベルで表す

よりも、

- **経済条件の束**
- **将来の支払いルール**の束

として表します。

たとえばスワップなら、「スワップ」という1個のオブジェクトを持つというより、

- 固定レッグの payout
- 変動レッグの payout
- 追加の bonus coupon payout
- upfront settlement payout

のように、**複数の payout を組み合わせて商品を作る**発想が自然です。

2.3 まず契約を表現し、その後のイベントは別で考える

CDM では、契約表現と、契約後に起きる出来事は別の関心事です。

たとえば、

- 新規約定
- 部分解約
- novation
- exercise
- settlement 実行
- cashflow 確定

は、契約そのものとは別に管理されることが多いです。

このコードは、**そのうち「契約そのもの」だけ**を扱います。

つまり、

- 約定時点で何が合意されているか
- 商品がどんな経済条件を持つか

だけを表し、**契約後のイベント管理は意図的に省略**しています。

これは設計上かなり重要です。

3. このコードの設計方針

ファイル冒頭にもありますが、このコードの方針は次の通りです。

- **契約表現に集中する**
- **イベント管理は省略する**
- CDM っぽいトップレベル構造を残す
- Party / Counterparty / Party1 / Party2 の考え方を残す
- PriceQuantity / Measure / Schedule の考え方を残す
- KO やデジタル条件のようなものを、**独立 Option ではなく payout-local feature** としても付けられるようにする
- クーポンスワップ専用のモデルにはしない

つまり、「CDM の思想を Python で読みやすい形にしたもの」と思ってください。

4. 全体構造をまず見る

このコードの最も大事な骨格は、次の入れ子です。

```
Trade
└─ TradableProduct
    │ counterparties
    └─ product
        └─ NonTransferableProduct
            │ identifiers
            │ taxonomies
            └─ economic_terms
                └─ payouts
```

この構造を言葉でいうとこうです。

- **Trade**
実際の取引そのもの
- **TradableProduct**
「この取引で何を取引したのか」と「誰と誰の間か」を持つ
- **NonTransferableProduct**
双務契約型の商品本体
たとえばスワップのような bilateral な契約
- **EconomicTerms**
商品の経済条件全体
- **Payout**
将来の支払い・受渡しルールの一部

この「Payout を複数持てる」ことが、今回のモデルで最も重要です。

5. 「Payout で組み立てる」とはどういう意味か

たとえば、あるスワップに

- 通常の fixed coupon
- 追加の bonus coupon
- さらに bonus だけ FX barrier で KO

があるとします。

これを 1 個の巨大なクラスで持つと、どんどん複雑になります。

そこでこのコードでは、

- InterestRatePayout として base coupon
- InterestRatePayout として bonus coupon
- bonus coupon の方にだけ ContingentFeature

のように表せるようにしています。

つまり、

同じ payoff mechanics を持つものを 1 payout としてまとめる
条件付き feature はその payout にぶら下げる

という考え方です。

これは、契約を読むときにも、実装するときにも分かりやすいです。

6. どこまでが CDM で、どこからがこの実装 独自か

ここは大事なので、正直に書きます。

6.1 かなり CDM 的なところ

- Trade -> TradableProduct -> Product -> EconomicTerms -> Payout
- Party1 / Party2 に正規化した counterparty 表現
- Measure , MeasureSchedule , PriceSchedule , QuantitySchedule
- PriceQuantity
- Asset / Observable
- SettlementTerms

このあたりは、かなり CDM の設計思想に沿っています。

6.2 このコード独自に整理しているところ

- ContingentFeature
- TriggerCondition
- FeatureEffect
- payout に generic な feature を付けるやり方

ここは、「CDM の公開ドキュメントにそのまま安定形で載っている」ものをそのまま写したというより、

CDM の方向性に沿って Python で使いやすく整理した部分です。

特に、

- KO
- KI
- デジタル条件
- payout の停止
- payout の rate/notional の調整

を表すために、汎用的な feature レイヤを置いています。

7. 各レイヤの読み方

ここから、コードを上から順に説明します。

7.1 共通ヘルパー

```
Number = Union[int, float, Decimal]
```

数値入力を柔軟に受けるための型です。

```
def _to_decimal(...)
```

すべての数値を `Decimal` に寄せるための関数です。

金融計算の世界では、`float` をそのまま信じると意図しない丸めが入りやすいので、これは実務的に良い癖です。

```
def _require(...)  
def _require_exactly_one(...)
```

`dataclass` の整合性チェック用です。

CDM では `cardinality` や `one-of` 条件が重要ですが、それを Python 側で簡単に再現するための補助です。

7.2 Enum 群

Enum は「モデル内で値のゆれを減らす」ためのものです。

例:

- CounterpartyRole
- PriceType
- SettlementType
- ObservationOperator
- FeatureEffectType
- TriggerType

たとえば CounterpartyRole を Party1 / Party2 に固定しているのは、CDM 的に非常に重要です。

実務では会社名や口座名を直接向き判定に使うと、商品定義が当事者依存になります。そこで「誰が payer か」ではなく、まず Party1 / Party2 という正規化役割に落としてから、実際の会社情報は別で紐づけます。

7.3 Identifier / Taxonomy

```
@dataclass(frozen=True)
class Identifier:
```

識別子です。

issuer, value, identifier_type を持ちます。

例:

- LEI
- ISIN
- internal ID
- UTI

などを表せます。


```
@dataclass(frozen=True)
class Taxonomy:
```

分類情報です。

商品や資産にタグを付ける用途です。

7.4 Party レイヤ

Party

```
@dataclass(frozen=True)
class Party:
```

当事者そのものです。

複数の ID を持てるようにしてあります。

Counterparty

```
@dataclass(frozen=True)
class Counterparty:
    role: CounterpartyRole
    party: Party
```

ここが重要です。

Party そのものではなく、**Party に role を与えたもの**です。

つまり:

- Bank A が Party1
- Bank B が Party2

のように、実在の相手を role に写像します。

AncillaryParty

主たる2当事者以外の補助的関与者を置く場所です。

代理人、取引所、計算エージェントなどを将来的に置きやすいようにしています。

7.5 Date / Schedule レイヤ

AdjustableDate

調整可能な日付です。

ビジネスデイ調整前の日付と、調整ルールを持てます。

RelativeDateOffset

「〇営業日前」「〇ヶ月後」のような相対日付表現です。

AdjustableOrRelativeDate

CDM ではこの種の“絶対日付 or 相対日付”がよく出ます。

このクラスはその one-of を表しています。

Frequency

頻度を表します。

例:

- 3M
- 6M
- 1Y

CalculationPeriodDates

計算期間の範囲と頻度です。

クーポンレグの accrual schedule をイメージすると分かりやすいです。

PaymentDates

支払頻度と遅延日数です。

ResetDates

リセット頻度と fixing offset です。

変動金利系に効きます。

7.6 Measure / Schedule レイヤ

ここは CDM の思想を理解する上で重要です。

UnitType

値の単位です。

たとえば:

- currency = USD
- financial_unit = SHARE

のように使います。

ちょうど1種類だけ持つ設計です。

MeasureBase

```
value  
unit
```

「値」と「単位」をセットにした最も基本的な抽象です。

Measure

`value` が必須です。

単なる測定値、数量、倍率などに使えます。

DatedValue

日付付きの値です。

将来時点ごとに値が変わる `schedule` を持つための部品です。

MeasureSchedule

ここがかなり大事です。

CDM では多くのものを「単発値」ではなく **スケジュール可能なもの**として扱います。

この MeasureSchedule は、

- 単一値としても持てる
- 日付付きの複数値としても持てる

ようにしています。

つまり、

- ノーショナルのステップ
- レートの変更
- quantity の変化

を素直に持てます。

PriceSchedule

価格の schedule です。

追加で、

- per_unit_of
- price_type
- price_expression

を持てます。

たとえば:

- USD per SHARE
- interest rate
- premium

のような意味付けができます。

QuantitySchedule / NonNegativeQuantitySchedule

数量の schedule です。

単位は必須で、負値は許しません。

multiplier を持てるので、

- 200 contracts
- each contract = 1000 bbl

のような表現に向いています。

7.7 Observable

```
@dataclass(frozen=True)
class Observable:
```

市場で観測されるものです。

例:

- USDJPY
- SOFR
- S&P 500
- 特定株価

ここではかなりシンプルにしていますが、概念としては

「価格や条件判定の対象になるもの」
です。

7.8 Settlement レイヤ

SettlementTerms

決済条件です。

- cash / physical
- settlement currency
- settlement date
- transfer settlement type

を持ちます。

BuyerSeller

PriceQuantity の決済方向を指定します。

これは option premium や FX forward のように、
「price/quantity そのものが settlement 対象」になる時に効きます。

7.9 PriceQuantity

これは CDM を理解するうえでかなり重要です。

```
@dataclass(frozen=True)
class PriceQuantity:
    prices
    quantities
    observable
    effective_date
    settlement_terms
    buyer_seller
```

これは、

- いくらか
- どれだけか
- 何に対する値か
- いつ有効か
- どう決済するか

を一塊にしたものです。

使いどころとしては例えば:

- option premium
- equity swap の underlying share quantity と initial price
- FX forward の amount/rate

などです。

7.10 Asset レイヤ

AssetBase

資産の識別・分類の土台です。

CashAsset

通貨そのもの。

InstrumentAsset

証券やデリバティブ原資産のようなものをざっくり表す箱です。

この実装では、Asset 部分は必要最小限に留めています。

7.11 RateSpecification レイヤ

FixedRateSpecification

固定レートを持つ payout 用です。

FloatingRateIndex

SOFR, LIBOR, TONA 的なインデックス表現用です。

FloatingRateSpecification

変動金利レック向けです。

index, spread, reset dates を持てます。

7.12 Feature / Trigger レイヤ

ここが、このコードの一番「設計を入れた」部分です。

ObservationTerms

何を、どう観測するかを表します。

- observable
- observation mode
- observation dates

などです。

TriggerLevel

トリガーの閾値です。

TriggerCondition

条件本体です。

- 何を観測するか
- どう比較するか
- 閾値はいくつか
- trigger type は何か

を表します。

FeatureEffect

条件が成立した時に何が起こるかです。

例:

- payout を止める
- 該当 period をゼロにする
- rate を下げる
- notional を減らす

ContingentFeature

trigger + effect をまとめたものです。

このレイヤの狙いは、

KO やデジタル条件を、必ずしも独立 Option としてではなく
payout に付く条件付き feature として表したい

ということです。

7.13 Payout レイヤ

ここが商品表現の中心です。

PayerReceiver

誰が払い、誰が受け取るかを
Party1 / Party2 で表します。

PayoutBase

全 payout の共通土台です。

共通で持つもの:

- payer_receiver
- price_quantities
- features
- payout_id
- description

InterestRatePayout

金利系 payout です。

持つもの:

- calculation periods
- payment dates
- notional schedule
- rate specification
- day count
- compounding

固定レグ、変動レグ、bonus coupon などのベースになります。

SettlementPayout

受渡し・決済そのものを表す payout です。

例:

- upfront payment
- premium payment
- FX amount exchange

OptionPayout

独立した option 的な payout 用です。

持つもの:

- call / put
- exercise terms
- underlier
- strike
- premium

今回のモデルでは用意していますが、

KO 付き coupon を表すのに必ずしもこれを使う必要はない

というのがポイントです。

7.14 EconomicTerms / Product / Trade レイヤ

EconomicTerms

複数の payout をまとめる場所です。

ここが「商品全体の経済条件」です。

NonTransferableProduct

双務契約型の商品。

スワップや OTC デリバティブをイメージすると分かりやすいです。

TradableProduct

商品本体と、2つの counterparties を束ねます。

ここで exactly 2 counterparties を強制しており、
しかも Party1 と Party2 が一度ずつ現れるようにしています。

Trade

最後に trade date や trade identifiers を持って、
実際の取引として完成します。

8. このコードの「一番大事な読み方」

このモデルを理解するうえで大事なのは、**Payout** をどう切るかです。

基本方針はこうです。

まず payoff mechanics で Payout を分ける

その上で、同じ contingent feature がかかるものに feature を付ける

たとえば:

- base coupon
- bonus coupon

- premium payment

は、普通は別 Payout です。

さらに bonus coupon にだけ KO があるなら、

- base_coupon: InterestRatePayout
- bonus_coupon: InterestRatePayout(features=(ko_feature,))

のようにします。

9. example_trade() を読む

このファイルの最後に、最小の組み立て例があります。

```
def example_trade() -> Trade:
```

これが README の実例として最重要です。

9.1 当事者を作る

```
party1 = Party(...)
party2 = Party(...)
cp1 = Counterparty(role=CounterpartyRole.PARTY_1, party=party1)
cp2 = Counterparty(role=CounterpartyRole.PARTY_2, party=party2)
```

まず実在の会社を Party として作り、それを Party1 / Party2 に割り当てています。

9.2 共通条件を作る

```
notional = flat_quantity(10_000_000, currency_unit("USD"))
fixed_rate = flat_price(...)
```

ノーションナルと金利を作っています。

`flat_quantity` と `flat_price` は convenience builder で、
README を読む段階では「よく使う定型を簡単に作るための関数」と思えば十分です。

9.3 観測対象と KO 条件を作る

```
fx_obs = Observable(name="USDJPY", ...)
ko_feature = ContingentFeature(
    trigger=TriggerCondition(...),
    effect=FeatureEffect(...)
)
```

ここで、

- 観測対象 = USDJPY
- 条件 = USDJPY >= 150
- effect = payout terminate

を 1 つの `ContingentFeature` にしています。

ここが「KO を独立 option にせず、payout に付く feature として表す」例になっています。

9.4 base coupon payout を作る

```
base_coupon = InterestRatePayout(...)
```

通常のクーポンです。

こちらには feature を付けていません。

9.5 bonus coupon payout を作る

```
bonus_coupon = InterestRatePayout(  
    ...,  
    features=(ko_feature,),  
)
```

こちらは bonus coupon で、KO feature が付いています。

つまり、

- base coupon
- bonus coupon

を別 Payout にしているわけです。

この切り方はかなり重要です。

9.6 商品全体を組み立てる

```
product = NonTransferableProduct(  
    economic_terms=EconomicTerms(  
        payouts=(base_coupon, bonus_coupon),  
        ...  
    )  
)
```

2つの payout を EconomicTerms に入れることで、
1つの商品を作っています。

ここが「商品を複数 payout の組み合わせで表す」実例です。

9.7 取引として完成させる

```
tradable_product = TradableProduct(...)
return Trade(...)
```

最後に counterparties と結びつけて Trade にしています。

10. 実際の使い方

ここからは、実務的に「このコードをどう使うか」です。

10.1 まずファイルを import する

```
from cdm_contract_model import *
```

または必要なものだけ個別 import します。

10.2 まず example_trade() を動かす

最初はこれが一番簡単です。

```
from cdm_contract_model import example_trade

trade = example_trade()

print(trade.trade_date)
print(trade.tradable_product.product.economic_terms.payouts)
```

これで、組み立て済みのオブジェクトが見られます。

10.3 自分で Party を作る

```
bank_a = Party(  
    party_ids=(Identifier(issuer="LEI", value="AAA"),),  
    name="Bank A",  
)  
  
bank_b = Party(  
    party_ids=(Identifier(issuer="LEI", value="BBB"),),  
    name="Bank B",  
)
```

10.4 Counterparty に役割を付ける

```
cp1 = Counterparty(role=CounterpartyRole.PARTY_1, party=bank_a)  
cp2 = Counterparty(role=CounterpartyRole.PARTY_2, party=bank_b)
```

10.5 Payout を1個作る

たとえば固定クーポンだけなら、こんな形です。


```

notional = flat_quantity(5_000_000, currency_unit("USD"))

fixed_rate = flat_price(
    value=0.03,
    unit=currency_unit("USD"),
    per_unit_of=currency_unit("USD"),
    price_type=PriceType.INTEREST_RATE,
)

fixed_coupon = InterestRatePayout(
    payout_id="fixed_leg",
    payer_receiver=PayerReceiver(
        payer=CounterpartyRole.PARTY_1,
        receiver=CounterpartyRole.PARTY_2,
    ),
    notional_schedule=notional,
    rate_specification=FixedRateSpecification(rate=fixed_rate),
    day_count_convention=DayCountConvention.ACT_360,
    calculation_period_dates=CalculationPeriodDates(
        effective_date=AdjustableOrRelativeDate(
            adjustable_date=AdjustableDate(date(2026, 1, 1))
        ),
        termination_date=AdjustableOrRelativeDate(
            adjustable_date=AdjustableDate(date(2031, 1, 1))
        ),
        frequency=Frequency(6, PeriodUnit.MONTH),
    ),
    payment_dates=PaymentDates(payment_frequency=Frequency(6, PeriodUnit.MONTH)),
)

```

10.6 Payout に feature を付ける

KO やデジタル条件を付けたい時は、ContingentFeature を作って payout に渡します。

```

fx_obs = Observable(
    name="USDJPY",
    asset_class=AssetClass.FX,
)

ko = ContingentFeature(
    name="CouponKO",
    trigger=TriggerCondition(
        observable=fx_obs,
        operator=ObservationOperator.GREATER_THAN_OR_EQUAL,
        level=TriggerLevel(value=Decimal("150"), unit=currency_unit("JPY")),
        trigger_type=TriggerType.KNOCK_OUT,
    ),
    effect=FeatureEffect(
        effect_type=FeatureEffectType.TERMINATE_PAYOUT,
        applies_to="this_payout",
    ),
)

```

そして payout に:

```

bonus_coupon = InterestRatePayout(
    ...,
    features=(ko,),
)

```

10.7 複数 Payout を EconomicTerms に入れる

```

economic_terms = EconomicTerms(
    payouts=(fixed_coupon, bonus_coupon),
    effective_date=...,
    termination_date=...,
)

```

これで商品全体の経済条件になります。

10.8 Product → TradableProduct → Trade と組み上げる

```
product = NonTransferableProduct(  
    identifiers=(Identifier(issuer="INTERNAL", value="MY-PRODUCT"),),  
    economic_terms=economic_terms,  
)  
  
tradable = TradableProduct(  
    product=product,  
    counterparties=(cp1, cp2),  
)  
  
trade = Trade(  
    trade_date=date(2026, 1, 1),  
    tradable_product=tradable,  
)
```

11. 典型的な設計パターン

11.1 固定レッグと変動レッグを持つスワップ

- fixed leg = InterestRatePayout
- floating leg = InterestRatePayout
- 両方を EconomicTerms.payouts に入れる

11.2 bonus coupon だけ KO

- base coupon = InterestRatePayout
- bonus coupon = InterestRatePayout(features=(ko_feature,))

この場合、KO の対象を「商品全体」ではなく
bonus payout だけに閉じ込められます。

11.3 premium payment

- SettlementPayout を使う

たとえば option premium や upfront fee を別 payout にできます。

11.4 独立した option

- OptionPayout を使う

ただし、KO 付き coupon を無理に OptionPayout に押し込む必要はありません。

12. このモデルで意識した「Payout の切り方」

このコードでは、次の考え方を採っています。

Payout はまず payoff mechanics で切る
contingent feature はその payout に付く

つまり、

- base coupon
- bonus coupon
- upfront
- premium

のように payoff の意味が違うものは、まず別 Payout です。

その上で、

- KO がある
- デジタル条件がある
- rate を下げる条件がある

といった conditional logic は features に入れます。

この順序の方が、設計が安定しやすいです。

13. バリデーションの意味

このコードは dataclass の `__post_init__` でいくつか条件をチェックしています。

例:

- Party は少なくとも1個の identifier を持つ
- UnitType は unit domain をちょうど1個だけ持つ
- MeasureSchedule は value か dated_values が必要
- QuantitySchedule は unit 必須かつ非負
- TradableProduct は counterparties がちょうど2人
- Party1 と Party2 が一度ずつ必要

これは、CDM の cardinality / one-of / basic consistency を Python で再現したものです。

つまりこのコードは、単なるデータ容器ではなく、
ある程度「それらしい契約オブジェクトしか作れない」ようにしてあります。

14. convenience builder について

ファイルの後半に、いくつか builder があります。

- currency_unit
- financial_unit
- decimal_measure
- flat_price
- flat_quantity

これらは、毎回 dataclass を深くネストして書くのを減らすためのものです。

たとえば

```
currency_unit("USD")
```

で `UnitType(currency="USD")` が作れます。

15. このコードでまだやっていないこと

このモデルは意図的に未完成な部分があります。

ここは README としてはむしろ大事です。

15.1 ライフサイクルイベント

入れていません。

- 部分解約
- novation
- exercise 実行
- settlement 実績
- cashflow 確定

は扱いません。

15.2 schedule 展開

`CalculationPeriodDates` は持っていますが、

- 実際に coupon schedule を生成する
- business day 調整する
- accrual factor を計算する

といった engine は入れていません。

15.3 trigger の実行判定

ContingentFeature は**条件そのもの**を表しますが、

- 実際に market data を当てて判定する
- どの period が止まるかを計算する

ところまではやっていません。

これはあえてです。

このファイルは contract model だからです。

15.4 CDM の全資産クラス

最小限の骨格だけです。

- equity
- credit
- commodity
- loan
- repo
- collateral

などの細かい型は入れていません。

15.5 FpML / CDM 完全互換

していません。

これは CDM inspired な Python 契約モデルであって、
公式 schema の 1 対 1 再現ではありません。

16. このコードをどう発展させると良いか

次にやるなら、私は次の順をおすすめします。

ステップ1

README を見ながら `example_trade()` を手で改造する

- 通貨を変える
- `rate` を変える
- `payout` を1個増やす
- `feature` を追加する

これが最短で理解が進みます。

ステップ2

schedule 展開ロジックを別モジュールで作る

この契約モデルはそのままにして、

- `accrual period` 生成
- `payment date` 生成
- `reset schedule` 生成

を別レイヤで実装するときれいです。

ステップ3

feature evaluation engine を別モジュールで作る

- `market observation` を入れる
- `trigger` 判定する
- `effect` を `contractual stream` に反映する

を別レイヤで作ると、contract model と clean に分離できます。

ステップ4

JSON シリアライズ層を作る

将来的には

- dataclass → dict
- dict → dataclass

を整えると、契約を保存・交換しやすくなります。

17. 読み方のおすすめ順

CDM に不慣れな人は、次の順で読むと入りやすいです。

1. `example_trade()`
2. `Trade` / `TradableProduct` / `NonTransferableProduct` / `EconomicTerms`
3. `PayoutBase` , `InterestRatePayout` , `SettlementPayout` , `OptionPayout`
4. `ContingentFeature` , `TriggerCondition` , `FeatureEffect`
5. `PriceQuantity` , `MeasureSchedule`
6. `Party` , `Counterparty`
7. `Date` / `Schedule` 周り

最初から上から順に全部読むより、この順の方がイメージがつかみやすいです。

18. 最後に一言でまとめる

このコードを一言で表すと、

CDM の契約表現の骨格を、Python の dataclass で読みやすく再構成したモデル

です。

特に重要なのは次の3点です。

- 商品は **複数の Payout の組み合わせ** で表す
- 当事者方向は **Party1 / Party2** に正規化する
- KO やデジタル条件は **payout-local feature** としても持てる

この3点が分かると、コード全体の見通しがかなりよくなります。

19. 付録: 最小サンプル

最後に、README だけ見て試せる最小例を載せます。

```

from datetime import date
from decimal import Decimal

from cdm_contract_model import (
    Party,
    Identifier,
    Counterparty,
    CounterpartyRole,
    NonTransferableProduct,
    TradableProduct,
    Trade,
    TradeIdentifier,
    EconomicTerms,
    InterestRatePayout,
    PayerReceiver,
    FixedRateSpecification,
    DayCountConvention,
    CalculationPeriodDates,
    AdjustableOrRelativeDate,
    AdjustableDate,
    Frequency,
    PeriodUnit,
    PaymentDates,
    flat_quantity,
    flat_price,
    currency_unit,
    PriceType,
    Observable,
    AssetClass,
    ContingentFeature,
    TriggerCondition,
    ObservationOperator,
    TriggerLevel,
    TriggerType,
    FeatureEffect,
    FeatureEffectType,
)

party1 = Party(
    party_ids=(Identifier(issuer="LEI", value="AAA"),),
    name="Bank A",
)

```

```

party2 = Party(
    party_ids=(Identifier(issuer="LEI", value="BBB"),),
    name="Bank B",
)

cp1 = Counterparty(role=CounterpartyRole.PARTY_1, party=party1)
cp2 = Counterparty(role=CounterpartyRole.PARTY_2, party=party2)

notional = flat_quantity(10_000_000, currency_unit("USD"))

rate = flat_price(
    value=0.02,
    unit=currency_unit("USD"),
    per_unit_of=currency_unit("USD"),
    price_type=PriceType.INTEREST_RATE,
)

usd_jpy = Observable(
    name="USDJPY",
    asset_class=AssetClass.FX,
)

ko_feature = ContingentFeature(
    name="CouponKO",
    trigger=TriggerCondition(
        observable=usd_jpy,
        operator=ObservationOperator.GREATER_THAN_OR_EQUAL,
        level=TriggerLevel(value=Decimal("150"), unit=currency_unit("JPY")),
        trigger_type=TriggerType.KNOCK_OUT,
    ),
    effect=FeatureEffect(
        effect_type=FeatureEffectType.TERMINATE_PAYOUT,
        applies_to="this_payout",
    ),
)

coupon = InterestRatePayout(
    payout_id="coupon_leg",
    payer_receiver=PayerReceiver(
        payer=CounterpartyRole.PARTY_1,
        receiver=CounterpartyRole.PARTY_2,
    ),
    notional_schedule=notional,

```

```

rate_specification=FixedRateSpecification(rate=rate),
day_count_convention=DayCountConvention.ACT_360,
calculation_period_dates=CalculationPeriodDates(
    effective_date=AdjustableOrRelativeDate(
        adjustable_date=AdjustableDate(date(2026, 1, 1))
    ),
    termination_date=AdjustableOrRelativeDate(
        adjustable_date=AdjustableDate(date(2028, 1, 1))
    ),
    frequency=Frequency(3, PeriodUnit.MONTH),
),
payment_dates=PaymentDates(payment_frequency=Frequency(3, PeriodUnit.MONTH)),
features=(ko_feature,),
)

product = NonTransferableProduct(
    identifiers=(Identifier(issuer="INTERNAL", value="PROD-XYZ"),),
    economic_terms=EconomicTerms(payouts=(coupon,)),
)

tradable = TradableProduct(
    product=product,
    counterparties=(cp1, cp2),
)

trade = Trade(
    trade_date=date(2026, 1, 1),
    tradable_product=tradable,
    trade_identifiers=(
        TradeIdentifier(identifier=Identifier(issuer="UTI", value="UTI-123")),
    ),
)

print(trade)

```

このサンプルでまず触ってみるのがおすすめです。